From POM to Binary: An Evidence-Based Assessment of GraalVM Native Image Readiness Using Maven Dependency Metadata

Sérgio Oliveira

x23170981@student.ncirl.ie

National College of Ireland | MSc Cloud Computer | Aug 2026

Supervisor: Punit Gupta

Who I Am

Sérgio Oliveira
BEng Manufacturing I MSc Cloud Computing

Manufacturing Engineer transitioning into Software Engineering and Cloud Computing.

Passionate about continuous learning and driven to design scalable cloud solutions as a future Solutions Architect.

https://www.linkedin.com/in/sergio-oliveira-063613163/

From Java Runtime to Native Executables

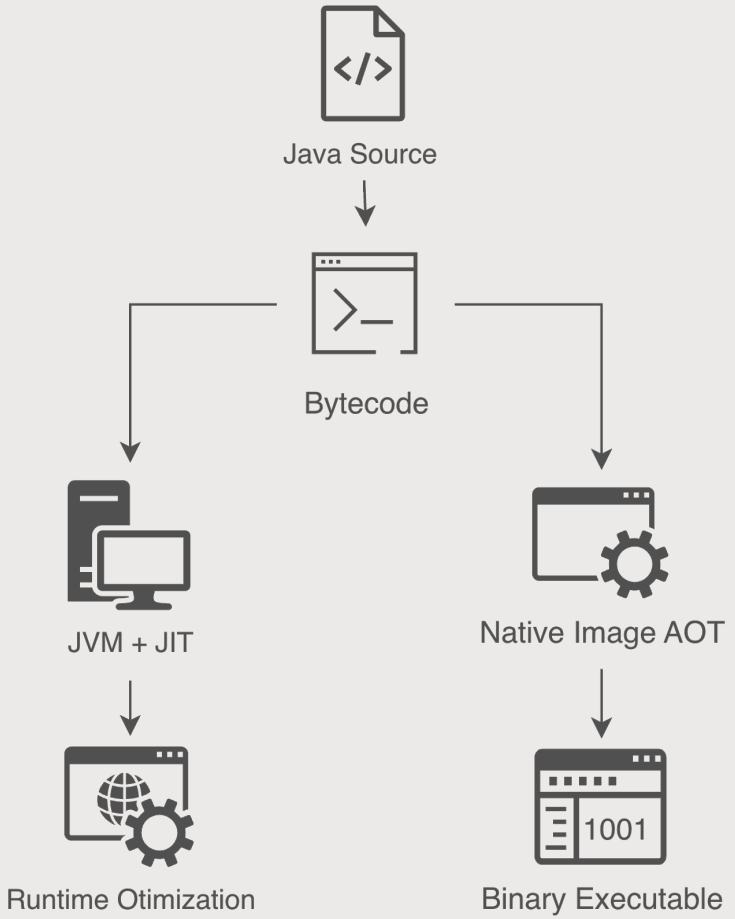
The JVM uses Just-In-Time compilation:

- bytecode is executed by the JVM
- runtime optimization occurs during execution
- dynamic features are naturally supported

Native Image follows a different model:

- ahead-of-time compilation
- closed-world assumption
- static analysis before execution

https://medium.com/@ayoubtaouam/understanding-how-java-code-runs-interpreter-jit-and-aot-explained-414eafd09159

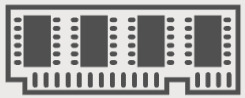


Why Native Image?

Native Image enables:

Faster

startup time



Lower

memory footprint

Smaller

deployment artifacts

Cloud

optimization

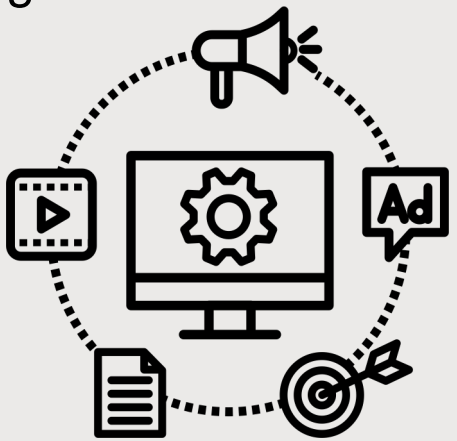
Native Image introduces a different deployment model optimized for specific scenarios.

https://www.graalvm.org/why-graalvm/

The Engineering Problem

Native Image Adoption Creates a New Migration Challenge

- The challenge is not generating a binary.
- The challenge is understanding whether an existing dependency ecosystem is prepared before compilation.



Large Maven ecosystem

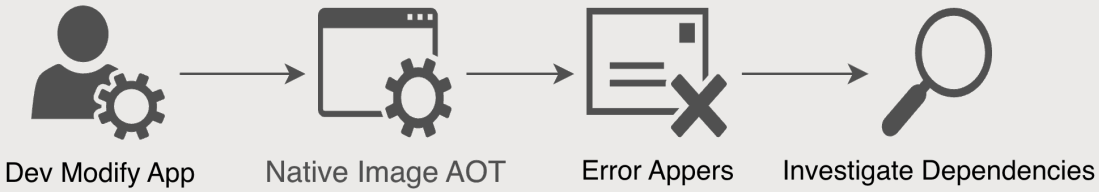
- Spring Boot
- Database
- Logging
- Security
- JSON
- Third-party libraries

Can this ecosystem support Native Image?

https://www.graalvm.org/22.0/reference-manual/native-image/

Current Practices

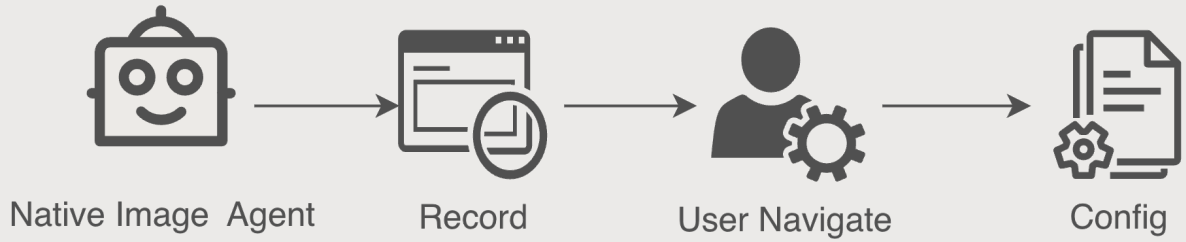
Compile Directly



Problems:

- Expensive compilation cycles
- Late discovery of incompatibilities
- Difficult diagnosis

Native Image Tracing Agent



https://www.graalvm.org/latest/reference-manual/native-image/metadata/AutomaticMetadataCollection/

Limitations:

- Runtime execution required
- Depends on execution coverage
- Configuration generated after implementation

Contribution based Shift-Left Opportunity

Can Dependency Analysis Provide Earlier Signals?

- Software Composition Analysis already provides visibility into dependency ecosystems.

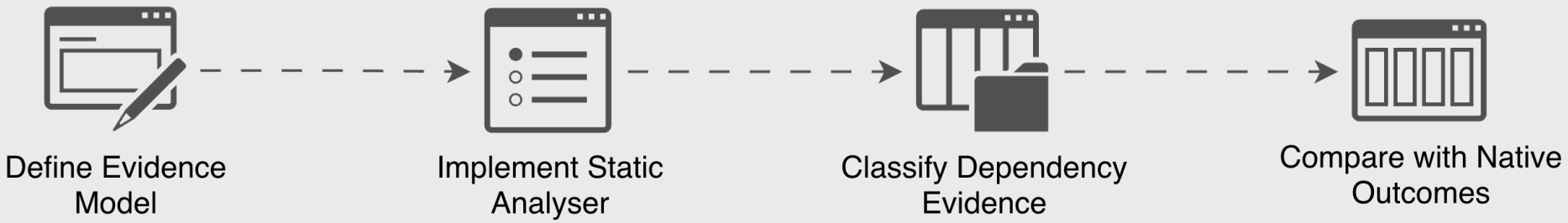
This work introduces a new perspective:

Native Image readiness can be investigated through structural evidence available in dependency metadata.

- Available Native Image metadata evidence
- Missing evidence signals

Research Question

To what extent can structural metadata signals
in Maven dependency graphs provide early evidence
of GraalVM Native Image readiness?

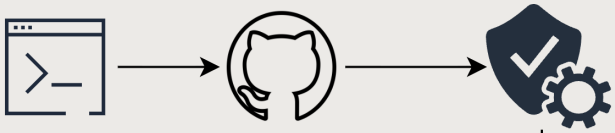


Framework

To investigate this research question, an evidence-based analysis framework was developed.

Submission & Validation

Submit GitHub repository URLs, perform basic validations, and create an asynchronous analysis task.



Graph Processor

Parse the POM, generate a CycloneDX SBOM, and build the dependency graph.

Component Analysis

Extract components, run native-image compatibility analysis, and classify direct vs transitive dependencies.

Results

Persist results, expose job status, and present concise summaries plus detailed AOT findings.

https://qba1p331pc.execute-api.eu-west-1.amazonaws.com/production

Case Studies

The framework was evaluated through four controlled case studies representing different dependency ecosystem scenarios.

Case	Purpose
Plain Java	Baseline
Modern Framework	Current Ecosystem
Legacy Framework	Migration Challenge
Hybrid Scenario	Mixed Dependency Ecosystem

Results

Framework successfully identified:

- Supported components
- Missing evidence
- Indirect evidence

The framework does not replace Native Image compilation.
It provides earlier structural visibility.

Limitations

- Does not analyze application reachability
- Does not reproduce GraalVM analysis
- Evidence availability depends on ecosystem metadata

Conclusion

Contribution to Knowledge:

- ✓ Native Image readiness can be investigated through structural dependency evidence.
- ✓ Software Composition Analysis concepts can support Native Image adoption beyond their traditional security-focused role.
- ✓ Evidence-based readiness assessment provides an earlier perspective that complements native compilation rather than predicting it.

Additional References

- Evaldsson, T. and M. Y. Bardakani (2024). ‘GraalVM’s Ahead-of-Time Compilation: Benefits and Challenges in Production’. Bachelor’s thesis. Karlskrona, Sweden: Department of Software Engineering, Blekinge Institute of Technology. DiVA id: diva2:1875531. In cooperation with Ericsson.

- Harrand, N. et al. (2022). ‘API beauty is in the eye of the clients: 2.2 million Maven dependencies reveal the spectrum of client-API usages’. In: Journal of Systems and Software 184. doi: 10.1016/j.jss.2021.111134. SJR: 0.950; Quartile: Q1; H-index: 141.

- Massacci, F. and I. Pashchenko (2021). ‘Technical leverage: Dependencies are a mixed blessing’. In: IEEE Security & Privacy 19(3), pp. 58–62. doi: 10.1109/MSEC.2021.3065627. SJR: 0.643; Quartile: Q1;H-index: 95.

- Nicho, M., I. Effiong and C. D. McDermott (2025). ‘Shift-left security: Integrating security in the initial phase of the DevOps methodology’. In: Proceedings of the 2025 9th International Conference on Cryptography, Security and Privacy (CSP). Okinawa, Japan: IEEE, pp. 182–191. doi: 10.1109/CSP66295.2025.00037. Google Scholar citations: 1.

- Pichler, C. et al. (2025). ‘Fast and compact: Reducing size of AOT-compiled Java code without sacrificing performance’. In: ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR). Singapore, 12-18 October 2025, pp. 12–22. doi: 10.1145/3759426.3760976. Google Scholar citations: 1.

The End

Sérgio Oliveira

BEng Manufacturing I MSc Cloud Computing